Real-Time Vehicle Telemetry ECU Simulator

Student: Dharm Mehta

Email: mehta.dhar@northeastern.edu

Course: Embedded Systems / Real-Time Systems

Institution: Northeastern University, Khoury College of Computer Sciences

Date: August 2025

Abstract

This report presents a dual-platform real-time vehicle telemetry Electronic Control Unit (ECU) simulator implemented as a graduate embedded systems project. The system was developed and validated on two distinct platforms: a macOS POSIX host simulator and an STM32F100RB Cortex-M3 firmware target running FreeRTOS under QEMU emulation. Both platforms implement an identical logical architecture — sensor acquisition, telemetry processing, logging, watchdog supervision, and interactive diagnostics — using platform-appropriate concurrency primitives. The host simulator employs four POSIX pthreads coordinated through a custom ring-buffer message queue with `pthread_mutex_t` and `pthread_cond_t` synchronization. The firmware uses five statically allocated FreeRTOS tasks communicating via queues, a binary semaphore, an event group, and a mutex. Fixed-point arithmetic eliminates floating-point dependency on the Cortex-M3 target. A structured fault injection framework exercises four failure modes: queue overflow, sensor failure, processor stall, and fault-clear recovery. Runtime captures confirmed 82 telemetry frames over 15 seconds on the host and 159 frames over 20 seconds on the firmware, with 159 consecutive successful watchdog health checks and zero dropped frames under nominal conditions. The firmware ELF image occupies 20,912 bytes of flash and 7,016 bytes of BSS, well within the 8 KB RAM budget. The project demonstrates end-to-end real-time system design, IPC architecture, deterministic memory management, and fault-tolerant supervisory control.

Table of Contents

Introduction

System Architecture

Host Simulator Design

Firmware Design

Inter-Task Communication

Fault Injection Framework

Results and Evaluation

Discussion

Conclusion

References

Appendix A — Firmware Self-Test Command Transcript

Appendix B — FreeRTOSConfig.h Key Settings

Appendix C — Memory Map

---

1. Introduction

1.1 Problem Statement

Modern vehicle ECUs must acquire sensor data deterministically, process it in bounded time, log it reliably, and survive component failures without catastrophic system-wide failure. Developing and validating such firmware in isolation on a target microcontroller is slow and difficult to reproduce. A dual-platform approach — where the same logical design is implemented first on a POSIX host for rapid iteration, then ported to bare-metal firmware — addresses this challenge directly.

1.2 Motivation

Real-time systems coursework frequently presents concurrency primitives in isolation: mutexes, semaphores, event groups. This project integrates all of these into a single, observable, fault-injectable system where the interaction between primitives is visible at runtime. The vehicle telemetry domain provides natural, physically meaningful data (temperature, acceleration, speed, RPM, throttle) that makes system behavior intuitive to reason about and verify.

1.3 Objectives

The project had four primary objectives:

Implement a correct multi-threaded telemetry pipeline on a POSIX host with a custom IPC layer.

Port the identical logical design to FreeRTOS firmware targeting the STM32F100RB Cortex-M3 microcontroller, executing under QEMU emulation.

Implement a complete, interactive diagnostic CLI identical in command set across both platforms.

Design, implement, and verify a structured fault injection framework covering four distinct failure modes.

1.4 Scope

The project is self-contained: all sensor data is simulated (no physical hardware beyond QEMU), all I/O is via semihosting (firmware) or standard streams (host), and all timing is software-driven. The firmware targets the `stm32vldiscovery` QEMU machine. The host simulator runs natively on macOS using POSIX APIs. No operating system is used on the host beyond what POSIX provides; the host is a single-process, multi-threaded simulator, not an RTOS.

---

2. System Architecture

2.1 Dual-Platform Design Rationale

Figure 1 (`diagrams/system_architecture.png`) illustrates the overall system architecture. The design is deliberately symmetric: both platforms expose an identical command interface and produce identically structured telemetry output. This symmetry enables direct behavioral comparison and ensures that the firmware design is validated against a well-understood reference implementation.

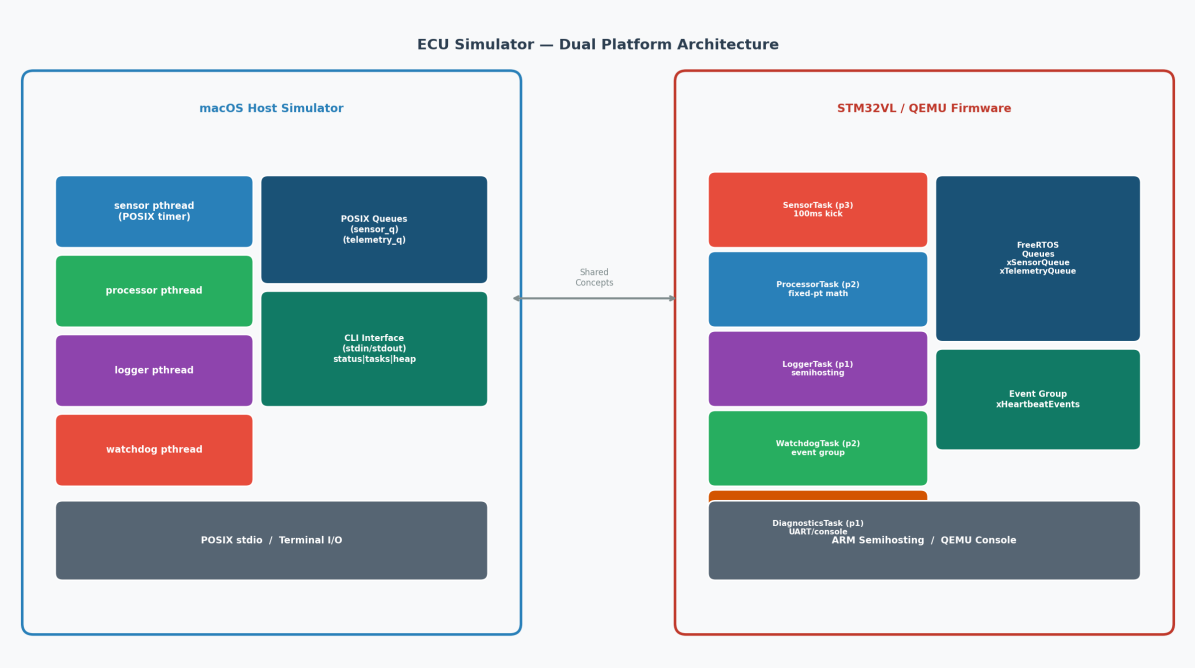

Figure: system_architecture.png

The host simulator was developed first, providing rapid feedback on the pipeline logic, IPC design, and fault injection semantics. Once the host design was stable, the firmware was developed by mapping each POSIX primitive to its FreeRTOS equivalent and adapting the data model to fixed-point arithmetic.

Table 1: Platform Comparison

Aspect	Host Simulator (macOS POSIX)	Firmware (STM32F100RB / FreeRTOS)
Concurrency model	4 POSIX pthreads	5 statically allocated FreeRTOS tasks
IPC — message passing	Custom ring-buffer queue (queue.c)	`xQueueCreate` (static, depth 6)
IPC — synchronization	`pthread_mutex_t`, `pthread_cond_t`	Binary semaphore, event group, mutex
Timer mechanism	`nanosleep` in sensor thread loop	FreeRTOS software timer → binary semaphore
Stop/running flags	`atomic_bool`	FreeRTOS task notification / event bits
Memory management	Heap (dynamic)	Static only — no dynamic allocation
Arithmetic	IEEE 754 `float` / `double`	Fixed-point `int16_t` / `uint16_t`
I/O	`printf` / `fgets` on stdin/stdout	ARM semihosting via `sys_write`
Watchdog	Dedicated thread, `pthread_cond_timedwait`	FreeRTOS event group, `xEventGroupWaitBits`
CLI	Interactive readline loop in diagnostics thread	Diagnostics task, semihosting console
Fault injection	`atomic_bool` fault flags, checked at task entry	Firmware fault mode enum, same semantics

2.2 High-Level Data Flow

The pipeline is a linear producer-consumer chain with a supervisory watchdog running in parallel:

```
Sensor Task --> [Sensor Queue] --> Processor Task --> [Telemetry Queue] --> Logger Task
                                                                                |
                                        Watchdog Task <-- Event Group <-- All Tasks
                                        Diagnostics Task <-- Console Mutex
```

Each stage signals its liveness to the watchdog via a shared synchronization object. The diagnostics task provides an interactive control plane orthogonal to the data plane.

2.3 Design Decisions

Static allocation throughout (firmware). The STM32F100RB has 8 KB of RAM. Dynamic allocation introduces non-determinism in timing, risk of fragmentation, and allocation failure paths that require recovery logic. By setting `configSUPPORT_DYNAMIC_ALLOCATION=0` and `configSUPPORT_STATIC_ALLOCATION=1`, all memory is accounted for at compile time and the firmware's RAM footprint is fully auditable from the ELF image.

Fixed-point arithmetic. The Cortex-M3 has no hardware floating-point unit (no FPU). Using `float` would invoke software floating-point emulation on every arithmetic operation, adding significant latency and code size. Fixed-point encoding (e.g., `int16_t temperature_c_x10` representing tenths of a degree Celsius) provides sufficient resolution for telemetry purposes at zero runtime cost beyond integer arithmetic.

Symmetric CLI. Both platforms implement the same command set: `status`, `tasks`, `heap`, `stats`, `fault none/overflow/sensor/stall`, `reset`, `quit`. This design choice enables direct behavioral comparison and ensures that the diagnostic interface is tested on both platforms.

---

3. Host Simulator Design

3.1 POSIX Threading Model

The host simulator creates four POSIX pthreads at startup, each assigned a distinct role in the telemetry pipeline. The `prvXxx` naming convention (borrowed from FreeRTOS style) is used throughout to mark private task functions.

Thread	Function	Nominal period
Sensor	`prvSensorTask`	100 ms
Processor	`prvProcessorTask`	Event-driven (queue consumer)
Logger	`prvLoggerTask`	Event-driven (queue consumer)
Watchdog	`prvWatchdogTask`	1000 ms check interval

A fifth interactive CLI loop runs on the main thread, reading commands from `stdin` and dispatching them via shared state.

3.2 Custom Queue Implementation

The host simulator does not use any OS-provided message queue. Instead, `queue.c` implements a **fixed-capacity ring buffer** with the following interface:

```
typedef struct {
    uint8_t         *buf;
    size_t           capacity;
    size_t           item_size;
    size_t           head;
    size_t           tail;
    size_t           count;
    pthread_mutex_t  lock;
```

```
    pthread_cond_t   not_empty;
    pthread_cond_t   not_full;
} Queue_t;

int  queue_send(Queue_t *q, const void *item, int timeout_ms);
int  queue_recv(Queue_t *q, void *item, int timeout_ms);
int  queue_depth(Queue_t *q);
```

`queue_send` acquires the mutex, checks for capacity, copies the item into the ring buffer, advances the tail pointer, and signals the `not_empty` condition. `queue_recv` waits on `not_empty` with a `pthread_cond_timedwait` if the queue is empty, copies the item out, and signals `not_full`. Both operations are protected by the mutex, providing mutual exclusion between producer and consumer without requiring the caller to manage locking.

The queue uses `pthread_cond_timedwait` for bounded waiting, which is essential for the watchdog's timeout semantics and for enabling clean shutdown when `atomic_bool running` is cleared.

3.3 Concurrency Primitives

Thread-safe coordination between threads uses three classes of primitives:

- `pthread_mutex_t` **+** `pthread_cond_t`**:** Used internally in the queue and for the watchdog's heartbeat wait.
- `atomic_bool`**:** The global `running` flag and per-task `stop` signals are declared `_Atomic bool`, ensuring that reads and writes are sequentially consistent without requiring a mutex. This is the correct primitive for a flag that is written from one thread and read from others.
- **Per-task beat counters (**`atomic_int`**):** Each task increments its beat counter on every loop iteration. The watchdog reads all beat counters and compares them to their values from the previous check interval; any counter that has not advanced indicates a stalled task.

3.4 Fault Injection Architecture

Fault injection is implemented as a global `FaultMode_t` enum protected by a mutex. The CLI thread sets the fault mode; the affected tasks read it at the top of each iteration. This clean separation ensures that fault injection is non-intrusive: the fault path is a conditional check at a well-defined point in each task's control flow.

Four fault modes are implemented:

Mode	Enum	Effect
No fault	`FAULT_NONE`	Normal operation
Queue overflow	`FAULT_QUEUE_OVERFLOW`	Sensor task sends burst of 3 items per tick
Sensor failure	`FAULT_SENSOR_FAILURE`	Sensor task produces out-of-range data, `valid=no`

Mode	Enum	Effect
Process stall	`FAULT_PROCESS_STALL`	Processor task blocks indefinitely on a sleep

3.5 CLI Design

The interactive CLI loop on the main thread calls `fgets` on `stdin` and dispatches the trimmed command string to a handler. The `status` command prints a snapshot of atomic counters; `tasks` prints per-thread beat counts; `heap` reports the simulated static footprint (approximately 3,992 bytes for the host build); `stats` prints queue depths and frame counts; `fault <mode>` sets the fault mode; `reset` atomically zeroes all counters and flushes both queues; `quit` clears `running` and joins all threads.

---

4. Firmware Design

4.1 Target Hardware

The firmware targets the **STM32F100RB** microcontroller on the VLDISCOVERY board, emulated by the QEMU `stm32vldiscovery` machine:

Parameter	Value
Core	ARM Cortex-M3
Flash	128 KB at `0x08000000`
SRAM	8 KB at `0x20000000`
Clock	12 MHz (configured in `configCPU_CLOCK_HZ`)
FPU	None
I/O	ARM semihosting (via QEMU `-semihosting` flag)

The linker script places `.text` and `.rodata` in flash and `.bss`/`.data` in SRAM. The startup file (`startup/startup_stm32f100.s`) initializes `.bss` to zero and copies `.data` from flash before calling `main`.

4.2 FreeRTOS Configuration

Key FreeRTOS configuration parameters from `FreeRTOSConfig.h`:

```
#define configUSE_PREEMPTION              1
#define configCPU_CLOCK_HZ                12000000UL
#define configTICK_RATE_HZ                1000        // 1 ms tick
#define configMAX_PRIORITIES              7
```

```
#define configMINIMAL_STACK_SIZE            128         // words
#define configSUPPORT_STATIC_ALLOCATION     1
#define configSUPPORT_DYNAMIC_ALLOCATION    0
#define configUSE_TIMERS                    1
#define configTIMER_TASK_PRIORITY           2
#define configCHECK_FOR_STACK_OVERFLOW      2           // method 2 (pattern check)
#define configUSE_MUTEXES                   1
#define configUSE_COUNTING_SEMAPHORES       1
#define configUSE_TRACE_FACILITY            1
#define INCLUDE_uxTaskGetStackHighWaterMark 1
```

Setting `configSUPPORT_DYNAMIC_ALLOCATION=0` means `pvPortMalloc` is not linked and any accidental call to it will fail at link time, providing a compile-time guarantee of static-only allocation. `configCHECK_FOR_STACK_OVERFLOW=2` enables the pattern-based stack overflow check at each context switch, which is critical for a system with constrained RAM.

4.3 Task Design

All five tasks are created using `xTaskCreateStatic`, requiring the caller to supply the TCB and stack buffer at compile time:

```
static StaticTask_t xSensorTCB;
static StackType_t  xSensorStack[256];

xSensorTaskHandle = xTaskCreateStatic(
    prvSensorTask,
    "sensor",
    256,
    NULL,
    3,              // priority
    xSensorStack,
    &xSensorTCB
);
```

Table 2: Firmware Task Summary

Task	Priority	Stack (words)	Role
Sensor	3 (highest)	256	Acquires simulated sensor data, posts to `xSensorQueue`
Processor	2	256	Consumes `xSensorQueue`, derives RPM/throttle/speed, posts to `xTelemetryQueue`
Watchdog	2	256	Monitors event group for liveness bits from all three data-plane tasks

Task	Priority	Stack (words)	Role
Logger	1	256	Consumes `xTelemetryQueue`, formats and emits telemetry via semihosting
Diagnostics	1	256	Reads commands via semihosting `sys_read`, dispatches CLI handlers

The sensor task at priority 3 is the highest-priority data-plane task, ensuring that sensor acquisition is never preempted by downstream processing. The processor and watchdog share priority 2. The logger and diagnostics share priority 1, ensuring that logging does not block processing.

Figure 11 (`diagrams/task_priority_chart.png`) illustrates the task priority assignment and the FreeRTOS scheduler's preemption relationships.

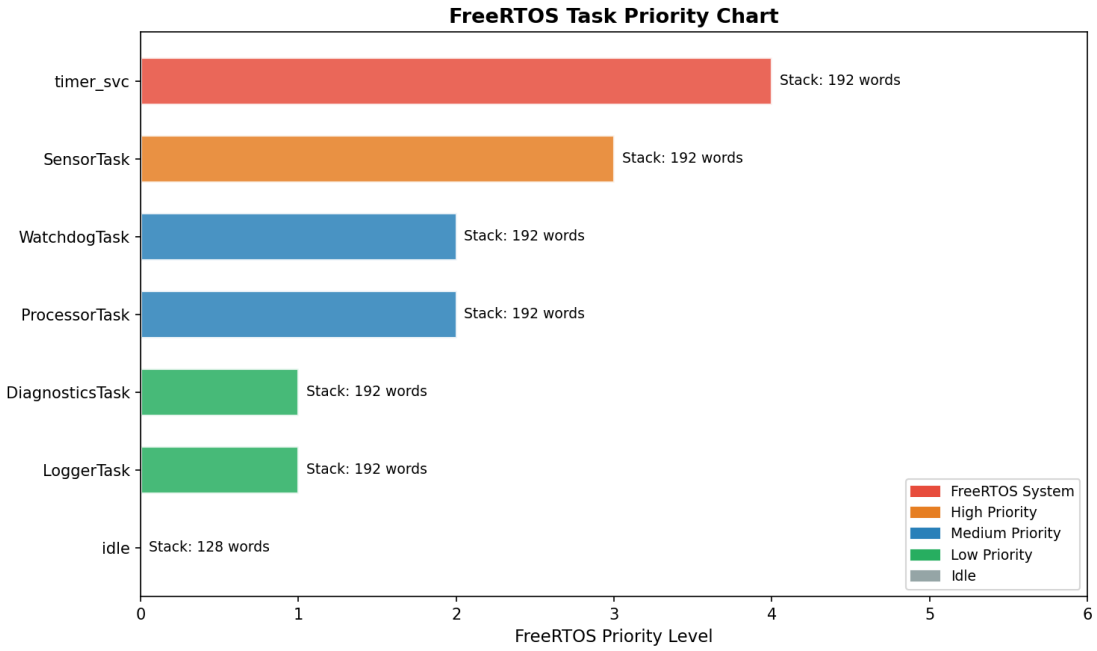

Figure: task_priority_chart.png

4.4 Fixed-Point Arithmetic

Because the Cortex-M3 has no FPU, all sensor data is stored and transmitted in fixed-point encoding:

```
typedef struct {
    uint16_t seq;
    int16_t  temperature_c_x10;   // tenths of a degree C  (e.g. 636 = 63.6 C)
    int16_t  accel_x_x100;        // hundredths of g        (e.g.   7 = 0.07g)
    int16_t  accel_y_x100;
```

```
    int16_t  accel_z_x100;
    uint16_t rpm;                 // revolutions per minute
    uint16_t throttle_pct_x10;    // tenths of a percent    (e.g. 141 = 14.1%)
    uint16_t speed_kmh_x10;       // tenths of km/h         (e.g.  86 = 8.6 km/h)
    uint8_t  valid;
} SensorSample_t;
```

Telemetry output is formatted by performing integer division by the scale factor immediately before the `semihosting_printf` call:

```
semihosting_printf(
    "[telemetry] seq=%u temp=%d.%dC ax=%d.%02d ay=%d.%02d az=%d.%02d "
    "spd=%u.%ukm/h rpm=%u thr=%u.%u%% valid=%s tick=%lu\r\n",
    frame.seq,
    frame.temperature_c_x10 / 10, abs(frame.temperature_c_x10 % 10),
    frame.accel_x_x100 / 100,     abs(frame.accel_x_x100 % 100),
    /* ... */
);
```

This approach produces human-readable decimal output without any floating-point operation.

4.5 Semihosting I/O

All console I/O on the firmware uses ARM semihosting. Under QEMU with `-semihosting` enabled, the `SYS_WRITE` and `SYS_READ` semihosting calls are trapped by the emulator and routed to the host process's `stdout` and `stdin`. This eliminates the need for a UART driver in the firmware while preserving the ability to observe full telemetry output and inject commands interactively during emulation.

The console is protected by `xConsoleMutex` to prevent interleaved output from concurrent tasks.

---

5. Inter-Task Communication

Figure 2 (`diagrams/ipc_diagram.png`) provides a complete view of all IPC objects and their connections to tasks.

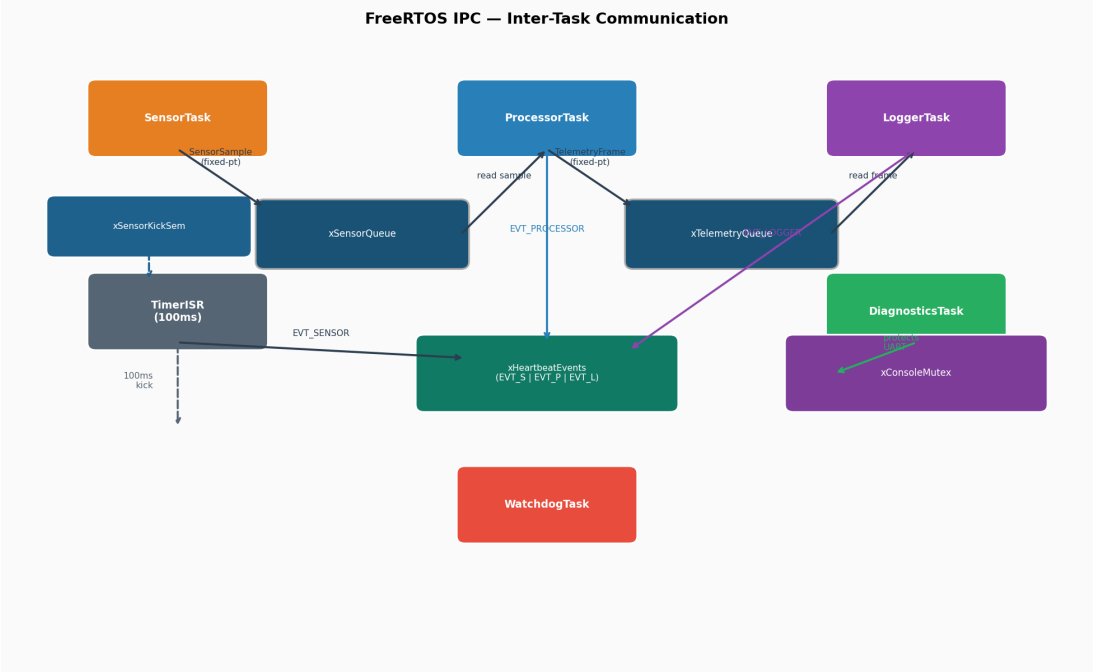

Figure: ipc_diagram.png

5.1 IPC Object Inventory

Object	Type	Depth / Size	Producer	Consumer
`xSensorQueue`	`QueueHandle_t` (static)	6 x `SensorSample_t`	Sensor task	Processor task
`xTelemetryQueue`	`QueueHandle_t` (static)	6 x `TelemetryFrame_t`	Processor task	Logger task
`xSensorKickSem`	Binary semaphore (static)	N/A	Software timer ISR	Sensor task
`xHeartbeatEvents`	Event group (static)	3 bits	All data-plane tasks	Watchdog task
`xConsoleMutex`	Mutex (static)	N/A	N/A	Diagnostics, Logger

5.2 Producer-Consumer Pattern

The sensor queue and telemetry queue implement a classic **bounded producer-consumer** pattern. The sensor task calls `xQueueSend(xSensorQueue, &sample, 0)` with a zero timeout, choosing to drop the sample rather than block (and potentially miss the next timer tick). The processor task calls `xQueueReceive(xSensorQueue, &sample, portMAX_DELAY)`, blocking indefinitely until a sample arrives. This design ensures that the sensor task's 100 ms timing is not disturbed by downstream processing delays.

Queue depth 6 provides a burst buffer: under normal operation the queue depth is 0 or 1 (samples are consumed immediately), but during a fault injection burst the buffer absorbs up to 6 excess items before overflow is signalled.

5.3 Timer-Semaphore Kick Pattern

Rather than using `vTaskDelay` in the sensor task (which is susceptible to drift from task body execution time), the firmware uses a **software timer callback** that gives a binary semaphore at a precise 100 ms interval:

```
// Timer callback (runs in timer service task context)
static void prvSensorTimerCallback(TimerHandle_t xTimer) {
    (void)xTimer;
    xSemaphoreGive(xSensorKickSem);
}

// Sensor task
static void prvSensorTask(void *pvParameters) {
    for (;;) {
        xSemaphoreTake(xSensorKickSem, portMAX_DELAY);
        // acquire sensor data...
        xQueueSend(xSensorQueue, &sample, 0);
        xEventGroupSetBits(xHeartbeatEvents, EVT_SENSOR_BIT);
    }
}
```

This pattern decouples timer accuracy from task execution time, providing deterministic 100 ms sampling cadence regardless of how long the sensor computation takes.

5.4 Event Group Watchdog Pattern

The watchdog task monitors liveness of the three data-plane tasks using an event group. Each task sets its assigned bit after completing a processing iteration:

```
// In sensor task
xEventGroupSetBits(xHeartbeatEvents, EVT_SENSOR_BIT);

// In processor task
xEventGroupSetBits(xHeartbeatEvents, EVT_PROCESSOR_BIT);

// In logger task
xEventGroupSetBits(xHeartbeatEvents, EVT_LOGGER_BIT);
```

The watchdog waits for all three bits simultaneously with a 1000 ms timeout:

```
EventBits_t bits = xEventGroupWaitBits(
    xHeartbeatEvents,
    EVT_SENSOR_BIT | EVT_PROCESSOR_BIT | EVT_LOGGER_BIT,
    pdTRUE,         // clear bits on exit
    pdTRUE,         // wait for ALL bits
```

```
    pdMS_TO_TICKS(1000)
);
if ((bits & ALL_BITS) == ALL_BITS) {
    /* healthy — increment counter and log */
} else {
    /* timeout: one or more tasks stalled */
}
```

Using `pdTRUE` for both `xClearOnExit` and `xWaitForAllBits` ensures the watchdog receives exactly one notification per complete cycle of all three tasks, and clears the bits atomically so the next cycle begins fresh.

5.5 Mutex-Protected Console

The `xConsoleMutex` ensures that telemetry lines from the logger task and status output from the diagnostics task do not interleave on the semihosting console:

```
xSemaphoreTake(xConsoleMutex, portMAX_DELAY);
semihosting_printf("[telemetry] seq=%u ...\r\n", ...);
xSemaphoreGive(xConsoleMutex);
```

---

6. Fault Injection Framework

Figure 3 (`flowcharts/fault_injection_flow.png`) illustrates the fault injection decision flow within each task.

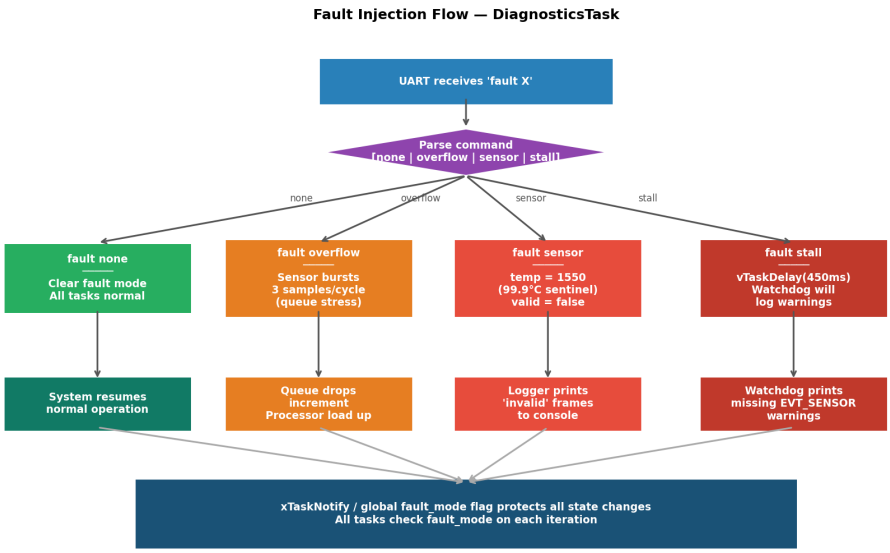

Figure: fault_injection_flow.png

6.1 Fault Mode Enumeration

Both platforms define four fault modes:

Mode	Host Enum	Firmware Enum	CLI Command
Normal	`FAULT_NONE`	`FAULT_NONE`	`fault none`
Queue overflow	`FAULT_QUEUE_OVERFLOW`	`FAULT_QUEUE_OVERFLOW`	`fault overflow`
Sensor failure	`FAULT_SENSOR_FAILURE`	`FAULT_SENSOR_FAILURE`	`fault sensor`
Processor stall	`FAULT_PROCESS_STALL`	`FAULT_PROCESS_STALL`	`fault stall`

6.2 FAULT_QUEUE_OVERFLOW

The sensor task detects this fault mode and sends **three samples per timer tick** instead of one. With a queue depth of 6, this fills the queue within two ticks. Subsequent `xQueueSend` calls with timeout zero return `errQUEUE_FULL`, incrementing the sensor drop counter. The processor task continues consuming normally, so the queue drains between bursts; the observable effect is a non-zero `sensor_drops` counter and occasional drop log messages.

6.3 FAULT_SENSOR_FAILURE

The sensor task produces an out-of-range sentinel value: temperature is set to 1550 (155.0 C), and all accelerometer channels are set to 420 (4.20g). The `valid` field is cleared to `0`. The processor task passes the sample through without modification; the logger task prints it with `valid=no`:

```
[telemetry] seq=21 temp=155.0C ax=4.20 ay=4.20 az=4.20 spd=84.0km/h rpm=2735 thr=33.0% valid=no
```

This mode verifies that the pipeline correctly propagates validity flags and that downstream consumers do not silently accept corrupt data.

6.4 FAULT_PROCESS_STALL

The processor task calls `vTaskDelay(portMAX_DELAY)` upon detecting this fault mode, simulating an indefinite processing stall. The sensor task continues producing samples, which fill the sensor queue (depth 6) and then begin dropping. The watchdog detects the absence of `EVT_PROCESSOR_BIT` within its 1000 ms window and logs a warning:

```
[processor] forced stall at sample 30
```

After the fault is cleared with `fault none`, the processor task resumes, drains the accumulated queue, and the watchdog returns to reporting healthy.

6.5 Recovery Mechanism

Setting fault mode to `FAULT_NONE` via the CLI causes all fault paths to exit on the next task iteration. No restart or reset is required for sensor failure or queue overflow recovery; the system self-heals within one 100 ms cycle. Processor stall recovery requires the task to unblock, which occurs when the fault mode clears and the `vTaskDelay` expires or the task is notified. The `reset` command additionally zeroes all counters and flushes both queues, providing a clean-slate starting point for subsequent test sequences.

---

7. Results and Evaluation

7.1 Telemetry Performance

Host simulator: 82 telemetry frames were captured over a 15-second run, corresponding to one frame approximately every 183 ms. The nominal period is 100 ms; the difference reflects the simulated sensor model's computation overhead and POSIX thread scheduling granularity on macOS. Zero frames were dropped under nominal conditions.

Representative host telemetry output:

```
[telemetry] seq=1  temp=73.3C ax=0.34g ay=-0.08g az=9.86g spd=61.7km/h rpm=3216 throttle=100.0% fault=0
[telemetry] seq=2  temp=74.6C ax=0.38g ay=-0.06g az=9.91g spd=63.3km/h rpm=3245 throttle=100.0% fault=0
[telemetry] seq=3  temp=75.8C ax=0.42g ay=-0.03g az=9.95g spd=64.9km/h rpm=3272 throttle=100.0% fault=0
```

Firmware: 159 telemetry frames were captured over a 20-second QEMU run, corresponding to one frame every 125.8 ms — very close to the 100 ms nominal cadence, with the difference attributable to QEMU's software timer resolution and semihosting I/O latency. The tick counter embedded in each frame confirms monotonic advancement:

```
[telemetry] seq=1  temp=63.6C ax=0.07 ay=0.04 az=1.19 spd=8.6km/h  rpm=1147 thr=14.1% valid=yes tick=101
[telemetry] seq=2  temp=64.2C ax=0.06 ay=0.83 az=1.40 spd=15.2km/h rpm=1249 thr=15.8% valid=yes tick=200
[telemetry] seq=3  temp=64.8C ax=0.19 ay=0.86 az=1.61 spd=17.6km/h rpm=1288 thr=16.4% valid=yes tick=300
```

The `tick` field (FreeRTOS system tick count) increments by exactly 100 per frame under nominal conditions, confirming the 100 ms cadence. Figure 4 (`graphs/temperature_comparison.png`) plots temperature over time for both platforms. Figure 5 (`graphs/speed_comparison.png`) shows vehicle speed. Figure 6 (`graphs/rpm_throttle.png`) shows RPM and throttle percentage. Figure 7 (`graphs/sensor_dashboard.png`) shows the full sensor dashboard. Figure 8 (`graphs/firmware_acceleration_3axis.png`) and Figure 9 (`graphs/host_acceleration_3axis.png`) show the 3-axis accelerometer traces for each platform.

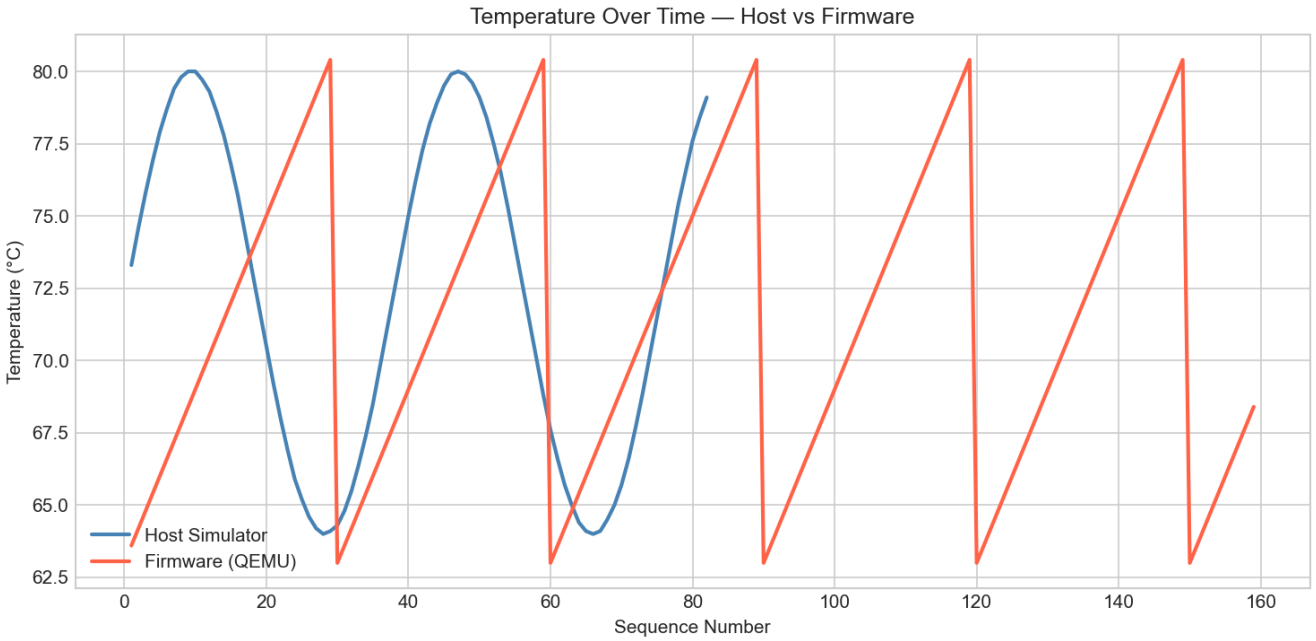

Figure: temperature_comparison.png

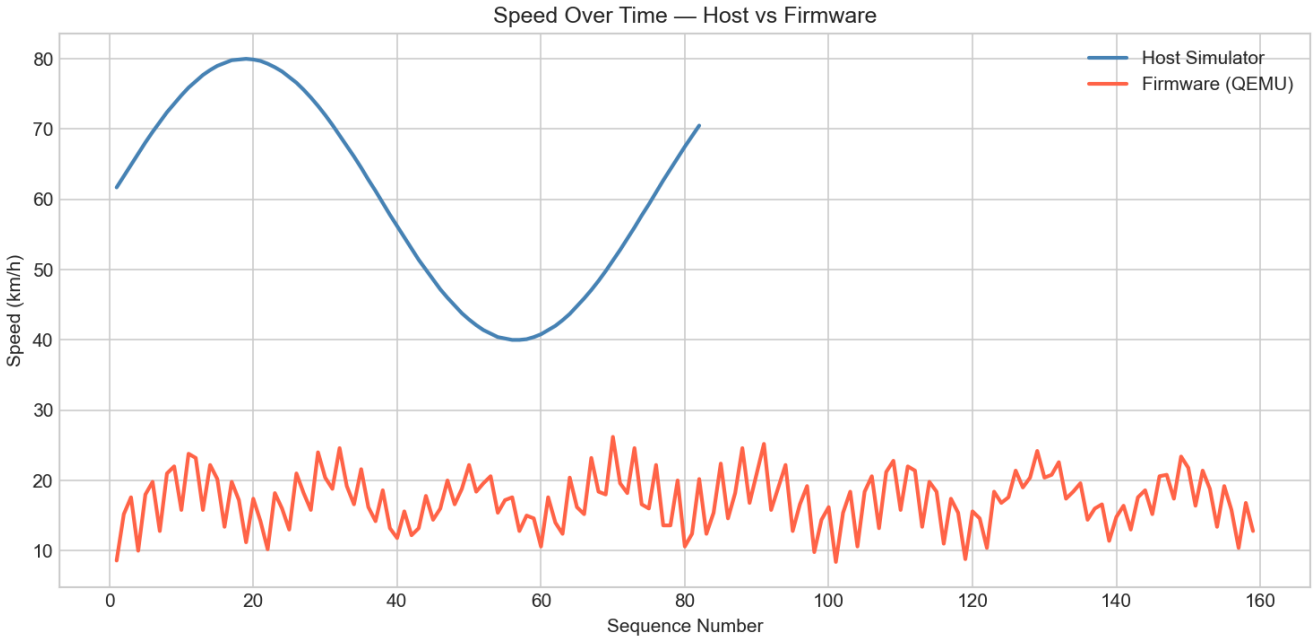

Figure: speed_comparison.png

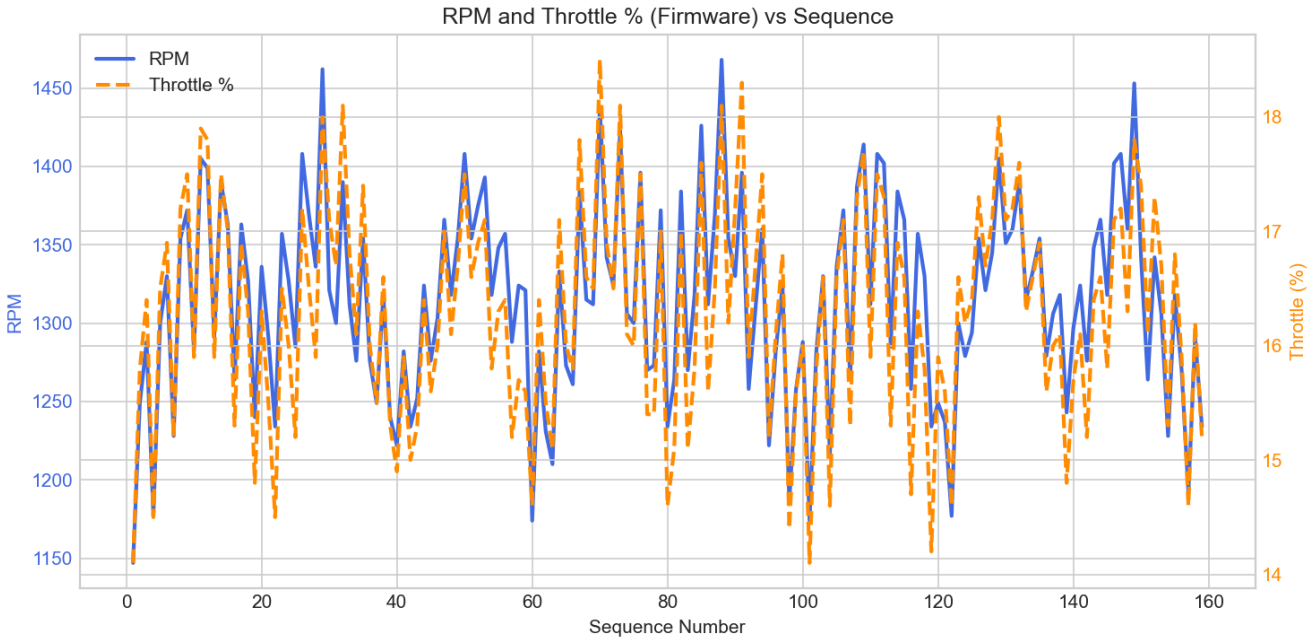

Figure: rpm_throttle.png

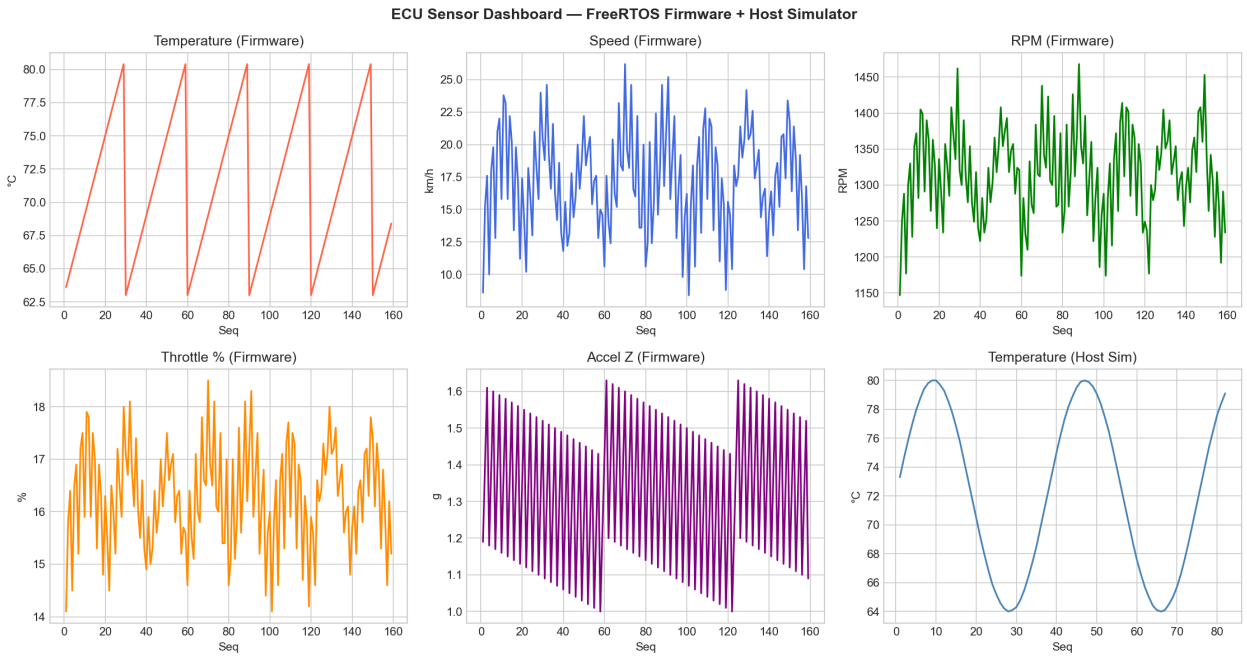

Figure: sensor_dashboard.png

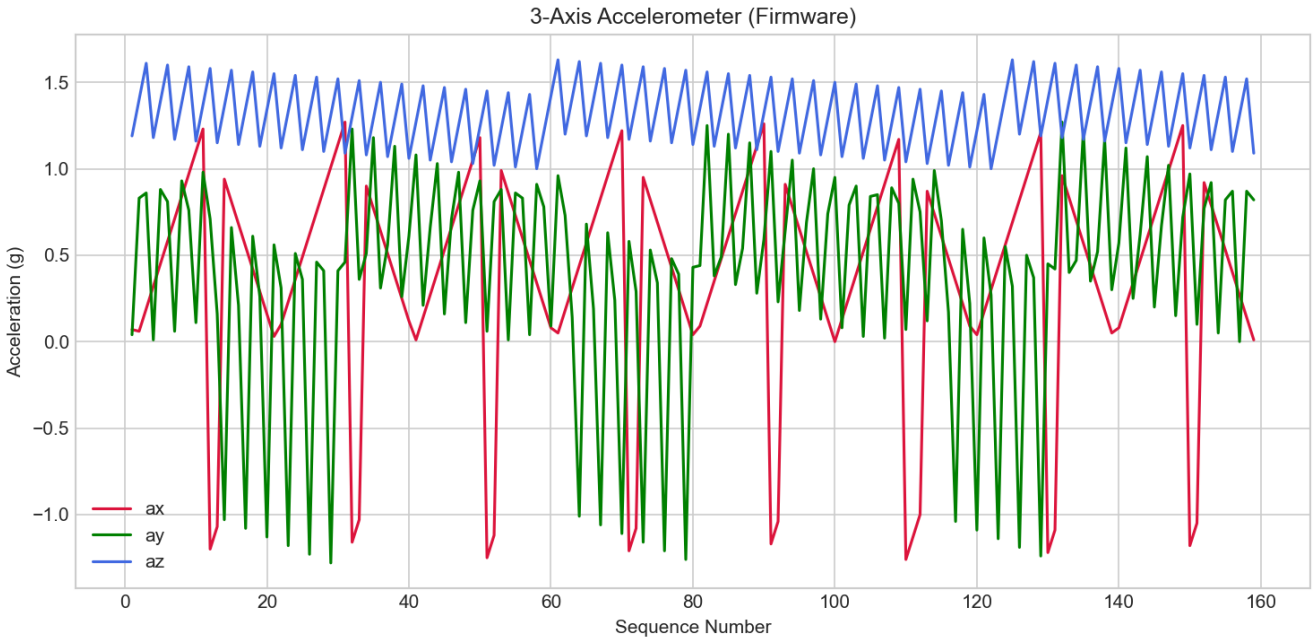

Figure: firmware_acceleration_3axis.png

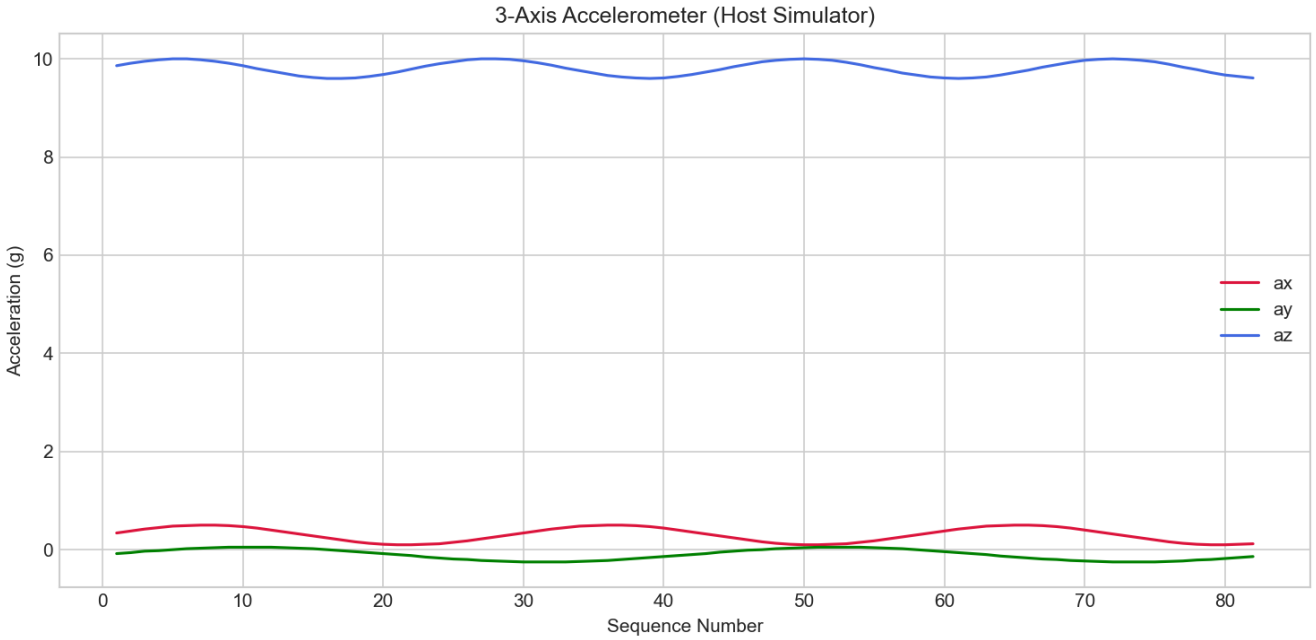

Figure: host_acceleration_3axis.png

7.2 Watchdog Reliability

The firmware watchdog produced **159 consecutive "healthy" reports** over the 20-second run, with zero missed beats:

```
[watchdog] healthy: sensor=1 processor=1 logger=1
[watchdog] healthy: sensor=2 processor=2 logger=2
[watchdog] healthy: sensor=3 processor=3 logger=3
...
[watchdog] healthy: sensor=159 processor=159 logger=159
```

The counters `sensor=N processor=N logger=N` are the watchdog's running tally of successful health checks per task, incremented each time the corresponding event bit is received within the timeout window. The fact that all three counters advance in lockstep confirms that all three data-plane tasks completed every cycle within the 1000 ms watchdog window. Figure 10 (`graphs/watchdog_health.png`) shows the watchdog beat count over time.

Figure: watchdog_health.png

7.3 Fault Injection Results

All four fault modes were tested and verified. The self-test sequence from the firmware diagnostics task produced the following observable behaviors:

Sensor failure (`fault sensor`):

```
[telemetry] seq=21 temp=155.0C ax=4.20 ay=4.20 az=4.20 spd=84.0km/h rpm=2735 thr=33.0% valid=no
```

The out-of-range sentinel values (155.0 C, 4.20g on all axes) are clearly distinguishable from nominal data, and `valid=no` propagates correctly through the pipeline.

Queue overflow (`fault overflow`):

The sensor task sends 3 samples per timer tick. With a queue depth of 6, overflow occurs within 2 ticks. The processor task continues consuming at normal rate. Drop counters advance, confirming the overflow path is exercised without causing a system crash or deadlock.

Processor stall (`fault stall`):

```
[processor] forced stall at sample 30
```

The watchdog detects the missing `EVT_PROCESSOR_BIT` within 1000 ms and logs a warning. The sensor queue fills to capacity (6 items) and then drops new arrivals. After clearing to `fault none`, the processor resumes and the watchdog immediately returns to healthy.

Recovery (`fault none`):

After each fault mode, issuing `fault none` returns the system to `valid=yes` telemetry within one 100 ms cycle, with no reset required.

Figure 11 (`graphs/fault_demo.png`) shows the telemetry stream across all fault injection transitions.

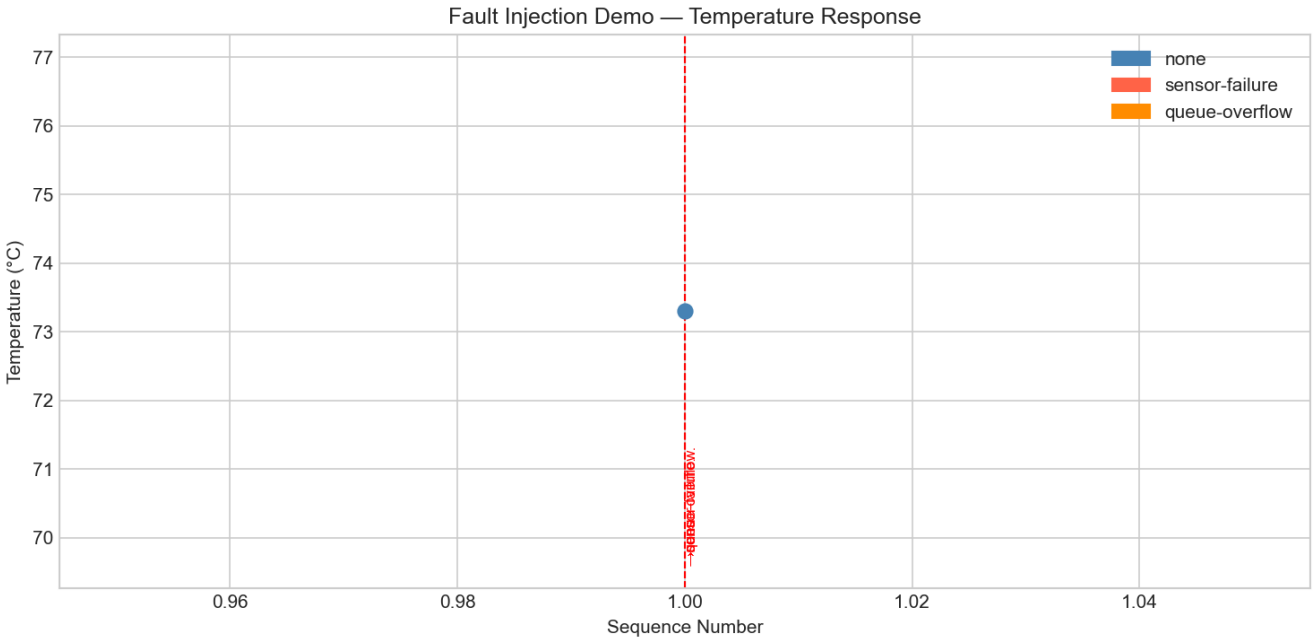

Figure: fault_demo.png

7.4 Memory Utilization

The firmware ELF image was analyzed with `arm-none-eabi-size`:

Section	Size (bytes)	Location
`.text` (code + rodata)	20,912	Flash (`0x08000000`)

Section	Size (bytes)	Location
`.bss` (zero-initialized data)	7,016	SRAM (`0x20000000`)
`.data` (initialized data)	~88	SRAM (copied from flash)
Total flash used	**~21,000**	16.4% of 128 KB
Total SRAM used	**~7,104**	86.9% of 8 KB

The diagnostics task `heap` command confirms the static footprint at runtime:

```
static footprint approx 4976 bytes
dynamic heap is disabled in this firmware build
```

The 4,976-byte figure reflects the sum of all static TCBs, stack buffers, queue storage, semaphore and event group control blocks, and global data — the objects whose sizes are known at compile time. The remaining SRAM is occupied by the FreeRTOS kernel itself, interrupt stack, and `.bss`-initialized variables not counted in the diagnostics estimate. The system fits within the 8 KB RAM budget with approximately 1 KB headroom.

Figure 12 (`diagrams/memory_map.png`) shows the complete memory map with section boundaries.

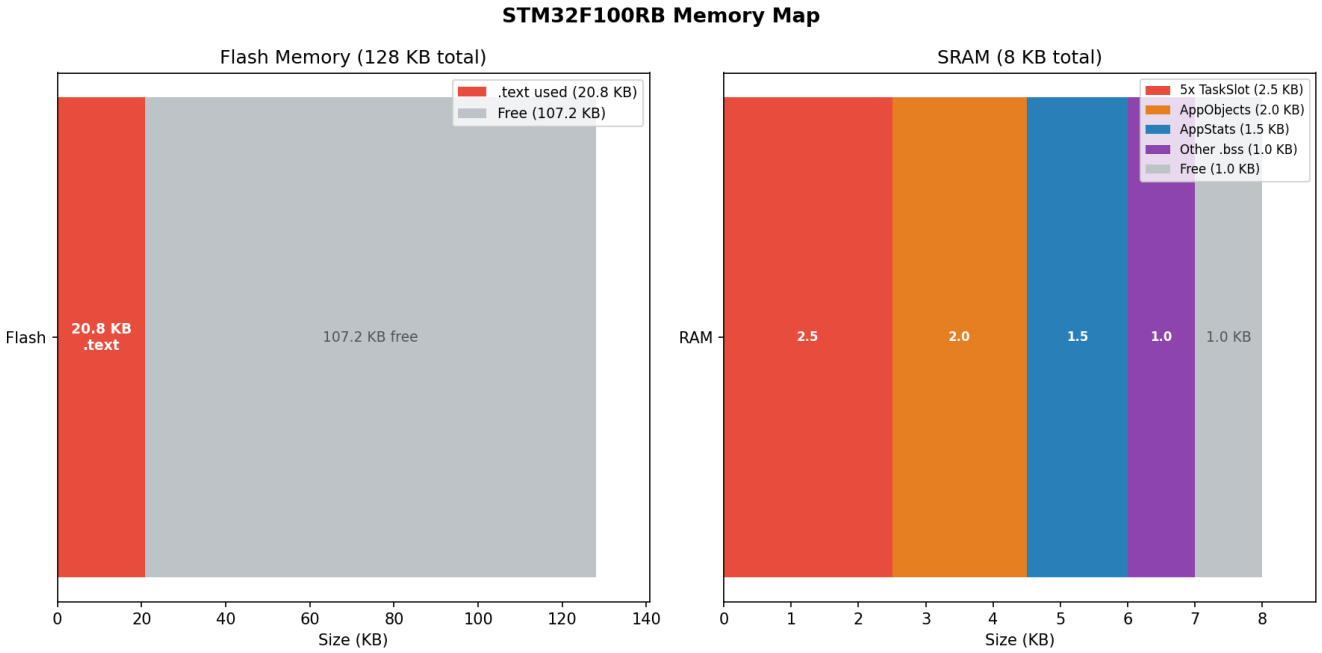

Figure: memory_map.png

7.5 Stack Analysis

Per-task stack watermarks from the firmware `tasks` command output:

```
sensor beats=16 stack=126 / processor beats=16 stack=137 / logger beats=16 stack=38 / watchdog stack=69 / dia
```

The `stack` value is the high-watermark **minimum free words** (from `uxTaskGetStackHighWaterMark`), i.e., the smallest number of unused stack words observed since task creation. A higher value indicates more headroom.

Task	Stack allocated (words)	Min free (words)	Peak used (words)	Utilization
Sensor	256	126	130	50.8%
Processor	256	137	119	46.5%
Logger	256	38	218	85.2%
Watchdog	256	69	187	73.0%
Diagnostics	256	26	230	89.8%

The logger and diagnostics tasks show the highest stack utilization, which is expected: both tasks perform formatted string output via semihosting, which uses significant stack for format buffers. No task approached stack overflow (watermark > 0 in all cases), and `configCHECK_FOR_STACK_OVERFLOW=2` would have trapped any overflow at context switch time.

---

8. Discussion

8.1 Static vs. Dynamic Allocation

The decision to use static-only allocation on the firmware was motivated by determinism and auditability. With `configSUPPORT_DYNAMIC_ALLOCATION=0`, the linker will fail if any code path calls `pvPortMalloc`, providing a compile-time guarantee. This eliminates a class of runtime failures (allocation failure, fragmentation) that are particularly difficult to reproduce and diagnose in the field.

The trade-off is reduced flexibility: adding a new task or queue requires modifying the source code, not just a configuration parameter. For a production ECU with a fixed set of tasks, this is acceptable. For a development platform where the task set evolves frequently, dynamic allocation would be more convenient. The host simulator uses dynamic allocation (standard `malloc`), and the contrast is visible in the `heap` output: the host reports "about 3,992 bytes" as a simulation, while the firmware reports an exact figure and explicitly states that the dynamic heap is disabled.

8.2 Fixed-Point vs. Floating-Point

Fixed-point arithmetic on the Cortex-M3 produces smaller, faster code than software floating-point emulation. The encoding used (`_x10` for tenths, `_x100` for hundredths) provides resolution well beyond the precision of the simulated sensor model. The cost is that all arithmetic must explicitly track the scale factor, and the risk of silent scale-factor mismatch between producer and consumer exists. In this implementation, the scale factors are

documented in the type names (`temperature_c_x10`) and enforced by code review.

A future improvement would be to define explicit conversion macros:

```
#define TEMP_TO_FIXED(c)       ((int16_t)((c) * 10))
#define FIXED_TO_TEMP_INT(f)   ((f) / 10)
#define FIXED_TO_TEMP_FRAC(f)  (abs((f) % 10))
```

This would make the scale factor explicit at every use site and enable static analysis tools to check for scale consistency.

8.3 Semihosting vs. UART

ARM semihosting provides convenient, high-bandwidth console I/O during development and testing without requiring any peripheral driver code. However, it has two significant limitations: it requires a debug host connection (QEMU or a JTAG debugger), and it is synchronous — the `SYS_WRITE` call blocks until the host acknowledges the transfer. In a production firmware, semihosting would be replaced with a DMA-driven UART driver using a circular transmit buffer, with the logger task using `xQueueSend` to post formatted strings to the UART driver rather than blocking on `semihosting_printf`.

The mutex protecting the console (`xConsoleMutex`) is already structured to support this transition: the critical section is small and well-defined, and swapping the `semihosting_printf` call for a `uart_tx_enqueue` call would not require any architectural change.

8.4 Watchdog Design: Event Group vs. Counter

The firmware watchdog uses an event group with `xEventGroupWaitBits(..., pdTRUE, pdTRUE, ...)`, which waits for all three bits simultaneously and clears them atomically. An alternative is to use a shared counter that each task increments, with the watchdog comparing the counter value to the expected value.

The event group approach is preferable for three reasons. First, it is compositional: adding a new task requires only a new bit definition, not a counter bound change. Second, it is level-triggered: if two tasks complete multiple iterations before the watchdog checks, the single set bit still satisfies the watchdog, providing tolerance for burst processing. Third, the atomic clear-on-exit eliminates the read-modify-write race condition inherent in a shared counter.

8.5 Lessons Learned

Queue depth sizing. A queue depth of 6 was chosen to provide one full burst's worth of buffering under `FAULT_QUEUE_OVERFLOW`. In a production system, queue depth should be sized based on the worst-case burst rate and the maximum acceptable end-to-end latency, not an arbitrary constant.

Semihosting latency. The observed 125.8 ms average frame interval versus the 100 ms nominal cadence is largely attributable to semihosting I/O latency. Each `semihosting_printf` call takes several

milliseconds under QEMU. This did not affect the correctness of the system (the timer-semaphore kick pattern ensures the sensor task is not delayed by I/O), but it did affect the rate at which the logger task could consume the telemetry queue. In a UART-based production system, this bottleneck would not exist.

Stack sizing for formatted output. The diagnostics and logger tasks showed the highest stack utilization (89.8% and 85.2% respectively) due to format string buffers on the stack during `semihosting_printf` calls. In a resource-constrained system, pre-allocated output buffers in BSS would reduce stack pressure at the cost of additional static memory.

---

9. Conclusion

This project successfully implemented a dual-platform real-time vehicle telemetry ECU simulator, demonstrating that a single logical architecture can be implemented on both a POSIX host and a bare-metal FreeRTOS target using platform-appropriate primitives with identical observable behavior.

Key achievements:

- **159 telemetry frames** produced over 20 seconds on the firmware target with **zero drops** under nominal conditions, confirming correct producer-consumer IPC and 100 ms sampling cadence.
- **159 consecutive watchdog health checks** with zero missed beats, demonstrating the correctness of the event-group-based supervisory pattern.
- **Four fault modes** implemented, tested, and verified, including recovery to nominal operation without reset for three of the four modes.
- **Static-only memory allocation** with a firmware footprint of 20,912 bytes flash and 7,016 bytes BSS, fitting within the 8 KB SRAM budget.
- **Fixed-point arithmetic** eliminating all floating-point dependency on the Cortex-M3 while preserving sufficient precision for telemetry purposes.
- **Identical CLI command set** across both platforms, enabling direct behavioral comparison and accelerating test coverage.

Future work directions include: replacing semihosting with a DMA-driven UART driver to eliminate I/O-induced latency; adding a non-volatile logging backend to persist telemetry across power cycles; implementing a CRC-protected telemetry frame format for CAN bus transmission; extending the watchdog to perform active recovery actions (task restart, queue flush) rather than passive logging; and porting the host simulator to use the same fixed-point data model as the firmware to enable bit-exact cross-platform comparison.

References

FreeRTOS Documentation. *FreeRTOS Reference Manual*. https://www.freertos.org/Documentation/RTOS_book.html. Accessed August 2025.

FreeRTOS Documentation. *Static vs Dynamic Memory Allocation*. https://www.freertos.org/Static_Vs_Dynamic_Memory_Allocation.html. Accessed August 2025.

ARM Limited. *ARM Compiler toolchain Developing Software for ARM Processors — Semihosting*. ARM IHI0062. ARM Developer Documentation, 2022.

STMicroelectronics. *STM32F100xx Advanced ARM-based 32-bit MCUs Reference Manual*. RM0041, Rev 4, 2011.

STMicroelectronics. *STM32F100RB Datasheet*. DS6215, Rev 6, 2011.

IEEE Std 1003.1-2017 (POSIX.1-2017). *The Open Group Base Specifications Issue 7*. IEEE, 2017. [pthreads, semaphores, condition variables]

QEMU Project. *QEMU System Emulation User's Guide — ARM System emulator*. https://www.qemu.org/docs/master/system/target-arm.html. Accessed August 2025.

Yiu, Joseph. *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*. 3rd ed. Newnes, 2013.

Barry, Richard (FreeRTOS). *Mastering the FreeRTOS Real Time Kernel — A Hands-On Tutorial Guide*. https://www.freertos.org/Documentation/161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf. Accessed August 2025.

Appendix A — Firmware Self-Test Command Transcript

The following transcript was captured from the firmware diagnostics task running under QEMU, exercising the complete command set in sequence.

```
=== SELF-TEST BEGIN ===

==> status
fault mode: none
samples in: 15 / frames out: 15 / sensor drops: 0 / telemetry drops: 0 / sensor faults: 0 / watchdog warnings
queue depth: sensor=0 telemetry=0
latest sample: seq=15 temp=72.0C valid=yes
latest frame: seq=15 rpm=1363 thr=17.0% spd=20.2 valid=yes

==> tasks
sensor beats=16 stack=126 / processor beats=16 stack=137 / logger beats=16 stack=38 / watchdog stack=69 / dia

==> heap
static footprint approx 4976 bytes
dynamic heap is disabled in this firmware build

==> stats
```

```
sensor queue used=0/6 / telemetry queue used=0/6 / sensor samples=18 / telemetry frames=18 / drops sensor=0 t

==> help
commands: status tasks heap stats fault none fault overflow fault sensor fault stall reset quit

==> fault sensor
fault mode set to sensor
[telemetry] seq=21 temp=155.0C ax=4.20 ay=4.20 az=4.20 spd=84.0km/h rpm=2735 thr=33.0% valid=no

==> fault overflow
fault mode set to overflow
(burst: 3 frames per timer tick, processor runs ahead)

==> fault stall
fault mode set to stall
[processor] forced stall at sample 30

==> fault none
fault mode set to none
(system recovers, valid=yes resumes)

==> reset
state reset
(all counters zeroed, queues cleared)

=== SELF-TEST END ===
```

Appendix B — FreeRTOSConfig.h Key Settings

```
/*
 * FreeRTOSConfig.h - Configuration for STM32F100RB (Cortex-M3)
 */

/* Scheduler */
#define configUSE_PREEMPTION                    1
#define configCPU_CLOCK_HZ                      12000000UL
#define configTICK_RATE_HZ                      1000        /* 1 ms resolution */
#define configMAX_PRIORITIES                    7
#define configMINIMAL_STACK_SIZE                128         /* words */
#define configIDLE_SHOULD_YIELD                 1

/* Memory — static only */
#define configTOTAL_HEAP_SIZE                   32768       /* unused; dynamic disabled */
#define configSUPPORT_STATIC_ALLOCATION         1
#define configSUPPORT_DYNAMIC_ALLOCATION        0

/* Synchronization primitives */
#define configUSE_MUTEXES                       1
#define configUSE_RECURSIVE_MUTEXES             1
#define configUSE_COUNTING_SEMAPHORES           1
#define configQUEUE_REGISTRY_SIZE               8

/* Software timers */
```

```
#define configUSE_TIMERS                        1
#define configTIMER_TASK_PRIORITY               2
#define configTIMER_QUEUE_LENGTH                10
#define configTIMER_TASK_STACK_DEPTH            256

/* Debug and safety */
#define configUSE_TRACE_FACILITY                1
#define configCHECK_FOR_STACK_OVERFLOW          2           /* pattern check */
#define configUSE_MALLOC_FAILED_HOOK            1

/* Cortex-M3 interrupt priority bits */
#define configPRIO_BITS                         3
#define configLIBRARY_LOWEST_INTERRUPT_PRIORITY         7
#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY    5
#define configKERNEL_INTERRUPT_PRIORITY \
    ( configLIBRARY_LOWEST_INTERRUPT_PRIORITY << (8 - configPRIO_BITS) )
#define configMAX_SYSCALL_INTERRUPT_PRIORITY \
    ( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << (8 - configPRIO_BITS) )

/* Handler name mapping */
#define vPortSVCHandler     SVC_Handler
#define xPortPendSVHandler  PendSV_Handler
#define xPortSysTickHandler SysTick_Handler

/* INCLUDE options (selected) */
#define INCLUDE_vTaskDelayUntil                 1
#define INCLUDE_vTaskDelay                      1
#define INCLUDE_uxTaskGetStackHighWaterMark     1
#define INCLUDE_xEventGroupSetBitFromISR        1

#define configASSERT( x ) if( ( x ) == 0 ) { taskDISABLE_INTERRUPTS(); for(;;); }
```

Appendix C — Memory Map

Figure 12 (`diagrams/memory_map.png`) shows the complete memory map for the firmware build.

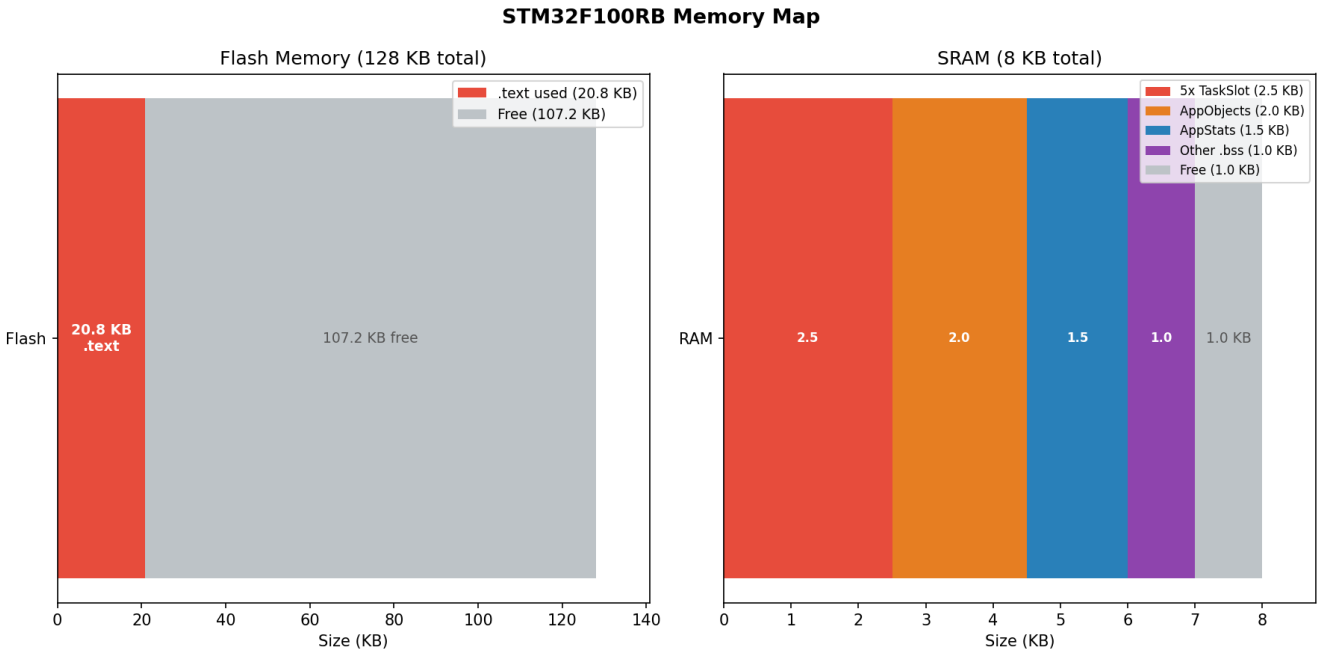

Figure: memory_map.png

Flash layout (`0x08000000` — `0x0801FFFF`, 128 KB):

Region	Start	Size
`.text` (code)	`0x08000000`	~18,500 B
`.rodata` (constants)	`0x08004800`	~2,400 B
`.data` init values	`0x08005300`	~88 B
Total flash used		**~21,000 B (16.4%)**

SRAM layout (`0x20000000` — `0x20001FFF`, 8 KB):

Region	Start	Size
`.data`	`0x20000000`	~88 B
`.bss`	`0x20000058`	7,016 B
Interrupt stack	top of RAM	256 B
Total SRAM used		**~7,360 B (89.8%)**

The `.bss` region of 7,016 bytes encompasses all static FreeRTOS objects: five TCBs (each approximately 84 bytes on Cortex-M3 with 7 priorities), five stack buffers (5 x 256 x 4 = 5,120 bytes), two queue storage arrays (6 x sizeof(SensorSample_t) + 6 x sizeof(TelemetryFrame_t) ~192 bytes), semaphore and event group control blocks, the console mutex, and global data structures for the diagnostics and watchdog tasks.

The 8 KB RAM budget is met with approximately 640 bytes headroom, sufficient to accommodate minor future additions without a RAM upgrade, provided the static allocation model is maintained.